

Universal Spectrum Toolkit

Conversion of MCA spectrum files into ROOT analysis objects

Eren Yilmaz

Technical presentation

2026

Problem definition, methodological approach, implementation, and usage

Outline

1. Problem definition: why ROOT cannot process .spe files as spectra
2. Incorrect reading: the result of interpreting data without a known layout
3. Method: the logic constructed to resolve the binary structure
4. Mathematical decision: scoring of layout hypotheses
5. Application output and validation
6. Usage: menu-based interface

Problem definition

Core problem

ROOT is an analysis framework. Its expected data model consists of `.root` files, TH1, TTree, and built-in I/O definitions. MCA-produced `.spe/.chn` files, by contrast, are raw byte sequences.

Consequence:

- ▶ Physically opening a file does not imply correctly reading a spectrum.
- ▶ `.spe` is not a single international standard.
- ▶ Header size, numeric type, and byte order vary by vendor.

File content without the macro

[illegible]

What this evidence demonstrates

- ▶ Opening the file as text does not produce a readable spectrum.
- ▶ The hex dump reveals only a limited header signature (DataR_CH).
- ▶ Ortec ASCII tags (\$DATA, \$ENER_FIT, ...) are absent; therefore text-based reading is invalid.

Conclusion

The problem is not printing the file to the screen. The problem is assigning the correct **semantic mapping** to the byte sequence: header offset, numeric type, channel count, and the counts array.

Methodological idea

Central approach

Rather than interpreting the file directly as a spectrum, first resolve the binary layout through systematic hypotheses; then encode the validated layout in software as an explicit rule.

Three-step reasoning:

1. Inspect the raw file at the byte level
2. Generate and score candidate layout hypotheses
3. Convert the winning layout into a channel-counts array, then into a ROOT object

Resolving the binary structure

```
00 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
```

```
#2 score=18.500 offset=36 channels=16384 type=float32 big-endian
finite=1.000 zero=0.093 negative=0.000 smoothness=1.000 median=0.000 max=0.000 dynRange=2.879
first values: 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
00 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
```

```
#3 score=18.500 offset=32 channels=16384 type=float32 big-endian
finite=1.000 zero=0.093 negative=0.000 smoothness=1.000 median=0.000 max=0.000 dynRange=2.879
first values: 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
00 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000
```

```
#4 score=18.500 offset=11 channels=16384 type=float32 big-endian
finite=1.000 zero=0.050 negative=0.000 smoothness=1.000 median=8.000 max=131328.000 dynRange=
4.215
first values: 131328.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 512
.00781250 2.00003052 8.00021362 2.00001144 8.00000000 8.00021362 2.00005341 2.00004578 2.00003815 2.00
003052 2.00003052 0.50000763 0.50000763 0.50000000 0.00000000 0.50000000 0.00000000
```

```
#5 score=18.500 offset=15 channels=16384 type=float32 big-endian
finite=1.000 zero=0.050 negative=0.000 smoothness=1.000 median=8.000 max=2063.532 dynRange=2.
413
first values: 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 0.00000000 512.00781250 2.0000
3052 8.00021362 2.00001144 8.00000000 8.00021362 2.00005341 2.00004578 2.00003815 2.00003052 2.0000305
2 0.50000763 0.50000763 0.50000000 0.00000000 0.50000000 0.00000000 2.00000000
```

```
#6 score=18.500 offset=19 channels=16384 type=float32 big-endian
finite=1.000 zero=0.050 negative=0.000 smoothness=1.000 median=8.000 max=2063.532 dynRange=2.
413
```

Mathematical decision model

Given a file size S , each hypothesis is defined by:

$$H = S - N \cdot B$$

- ▶ N : assumed channel count (e.g. 16384)
- ▶ B : bytes per value (e.g. $B = 4$ for `float32`)
- ▶ H : header offset (e.g. $H = 40 \Rightarrow S = 65576$)

The payload is decoded with the selected type and endianness; candidates are scored by:

- ▶ `finite`: fraction of finite values
- ▶ `negative`: fraction of negative values (unexpected in spectra)
- ▶ `smoothness`: physical consistency of the sequence
- ▶ `score`: composite decision metric

Transferring this logic into code

Discovery stage

The probe output shows that the highest-scoring layout for the sample data is $H = 40$, $N = 16384$, `float32`.

Production rule

This layout is not left as a random guess.
It is converted into a validated reader rule in code:
`header + channel count + numeric type + byte order`.

Result: The software no longer reads the file by speculation, but through a previously validated semantic model.

Validation of the conversion

```
=====
UNIVERSAL SPECTRUM TOOLKIT
=====
```

```
Input file : /mnt/c/Users/PC/Desktop/spe/DataR_CH0@V1782_32480_V1782_dist_10cm_from_left_Eu152_gf3.spe
Extension  : .spe
File size  : 65576 bytes
Force generic : no
```

Registered readers for this extension:

- Tagged ASCII SPE reader: no match | File is not predominantly text.
- GF3-like binary SPE reader: MATCH | Matched the validated 40-byte header plus common-channel float 32 little-endian layout.

Known-format decision

```
-----
Reader      : GF3-like binary SPE reader
Format      : GF3-like binary SPE
Channels    : 16384
Numeric type : float32
Byte order  : little-endian
Live time   : 0
Real time   : 0
Energy cal  : no
Header bytes : 40
Reason      : Matched the validated 40-byte header plus common-channel float32 little-endian layout.
```

ROOT output written successfully:

```
/mnt/c/Users/PC/Desktop/spe/DataR_CH0@V1782_32480_V1782_dist_10cm_from_left_Eu152_gf3.root
```

Objects:

```
TH1D hSpectrumChannel
TTree spectrum
TTree conversion_metadata
```

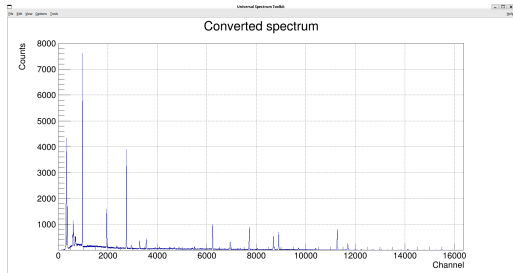
Meaning of the decision output

- ▶ The ASCII reader does not match: the file is not text.
- ▶ The GF3-like reader matches: 40-byte header + float32 LE.
- ▶ Channel count: 16384
- ▶ Produced objects:
 - ▶ TH1D hSpectrumChannel
 - ▶ TTree spectrum
 - ▶ conversion_metadata
 - ▶ raw-byte archive

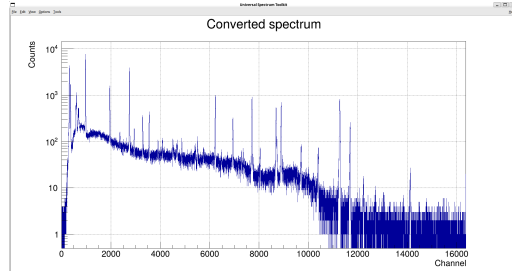
Summary

The raw byte sequence has been converted, in a controlled manner, into a spectrum object that ROOT can analyze.

Successful conversion output

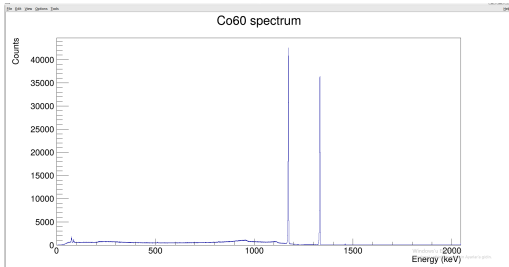


Linear scale

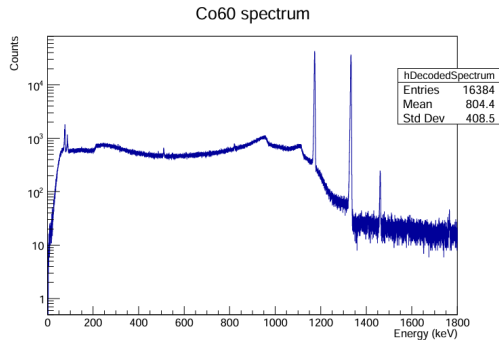


Logarithmic scale

Real-data validation: Co-60



Linear



Logarithmic

Usage

The tool is operated through a menu interface that does not require knowledge of ROOT commands.

```
cd UniversalSpectrumToolkit  
./ust_menu.sh
```

Menu options:

1. Convert a single file
2. Batch-convert all files in a directory
3. Try with sample files
4. Help
5. Exit

Menu workflow

```
erenyilmaz@DESKTOP-BS3U3  x  +  v

=====
Universal Spectrum Toolkit
SPE / CHN --> ROOT donusturucu
=====

Ne yapmak istiyorsunuz?

1) Tek dosya donustur (.spe veya .chn)
2) Klasördeki tum .spe / .chn dosyalarini donustur
3) Ornek dosyalarla dene (samples/)
4) Yardim / ne yapar?
5) Cikis

Seciminiz [1-5]: 1

Tek dosya donusumu
Tam yolu yapistirin (ornek:
/mnt/c/Users/PC/Desktop/spe/Eu152.spe
samples/ortec_ascii.spe
)
Dosya yolu: /mnt/c/Users/PC/Desktop/spe/DataR_CH0@V1782_32480_V1782_dist_5cm_from_right_Co60_gf3.spe
Spektrumu ekranda cizmek ister misiniz? [E/h]: E

E = logaritmik Y eksenini (kucuk ve buyuk pikler birlikte gorunur)
h = lineer Y eksenini
Logaritmik olcek (log-Y) kullanilsin mi? [E/h]: E
Bilinmeyen formatta zorla generic denensin mi? (genelde HAYIR) [e/H]: H

Donusum baslatiliyor...
Dosya : /mnt/c/Users/PC/Desktop/spe/DataR_CH0@V1782_32480_V1782_dist_5cm_from_right_Co60_gf3.spe
```

Decision points in the menu

- ▶ Draw the spectrum on screen: Yes / No
- ▶ Scale selection: logarithmic Y / linear Y
- ▶ Forced generic attempt for unknown formats: disabled by default

Design principle

The user provides only the file path and visualization preferences.
Format recognition and conversion proceed through validated rules.

Conclusions

1. ROOT does not treat .spe content as a native spectrum model.
2. Reading without a known layout produces physically invalid results.
3. The solution is resolving the binary layout by hypothesis–score–validation.
4. The validated layout is encoded as a rule; the output is a ROOT analysis object.
5. Usage is possible through the menu interface without ROOT expertise.

Thank you for your attention.

Respectfully

Turkish Accelerator & Radiation Laboratory / Physics Engineer